Interactive visualization in R with Shiny

HUGEN 2073

Genomic Data Visualization and Integration

Jon Chernus

Some material adapted from https://shiny.rstudio.com

Setup

Download the folder 16_shiny_data from this Canvas module.

It contains some Shiny apps, an R script, and a dataset.

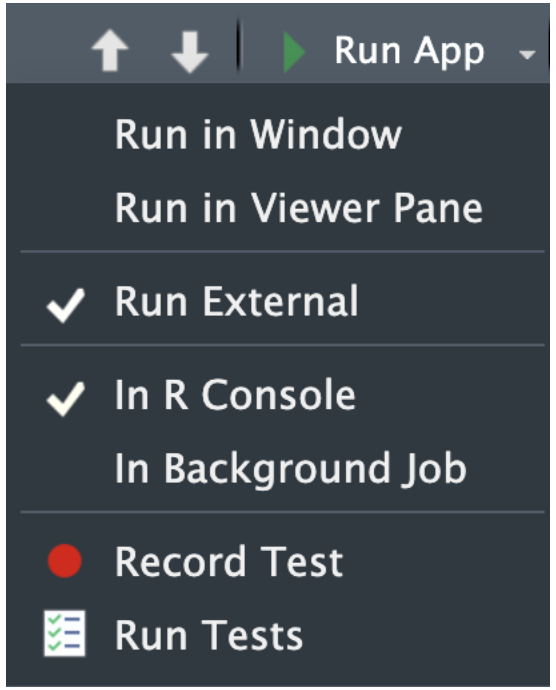
Shiny

- R package for building interactive web apps
- “Automatic ‘reactive’ binding between inputs and outputs and extensive prebuilt widgets make it possible to build beautiful, responsive, and powerful applications with minimal effort.”
- For genomics, perhaps most useful for data exploration?

What is a Shiny app?

Just an R script, with a specific structure.

Can be run in RStudio



Other details

- Usually name it something like "`app.R`"
- Put it alone in its own folder, say, "`app_dir`"
- Can run it with `runApp("/path/to/app_dir/")`
- Exit with the escape key
- Can open in RStudio viewer window or in a browser

Structure of a Shiny app

Three parts

- User interface object
- Server function
- A call to the `shinyApp` function

(You can also define global-level variables at the top, before the ui.)

The skeleton looks like this:

```
library(shiny)

# See above for the definitions of ui and server
ui <- ...

server <- ...

shinyApp(ui = ui, server = server)
```

More detailed structure of a Shiny app

This *is* a Shiny app. A completely blank one.

```
library(shiny)

# Define UI ----
ui <- fluidPage(

)

# Define server logic ----
server <- function(input, output) {

}

# Run the app ----
shinyApp(ui = ui, server = server)
```

ui

Nested structure wrapped in the function `fluidPage`.

ui specifies everything the user sees and interacts with

- Titles (`titlePanel`)
- Layout (`sidebarLayout`) always takes two arguments
  - Sidebar panel (the interactive control widgets; `sidebarPanel`)
    - The widgets you want are specified inside of `sidebarPanel`
  - Main panel (where output goes; `mainPanel`)

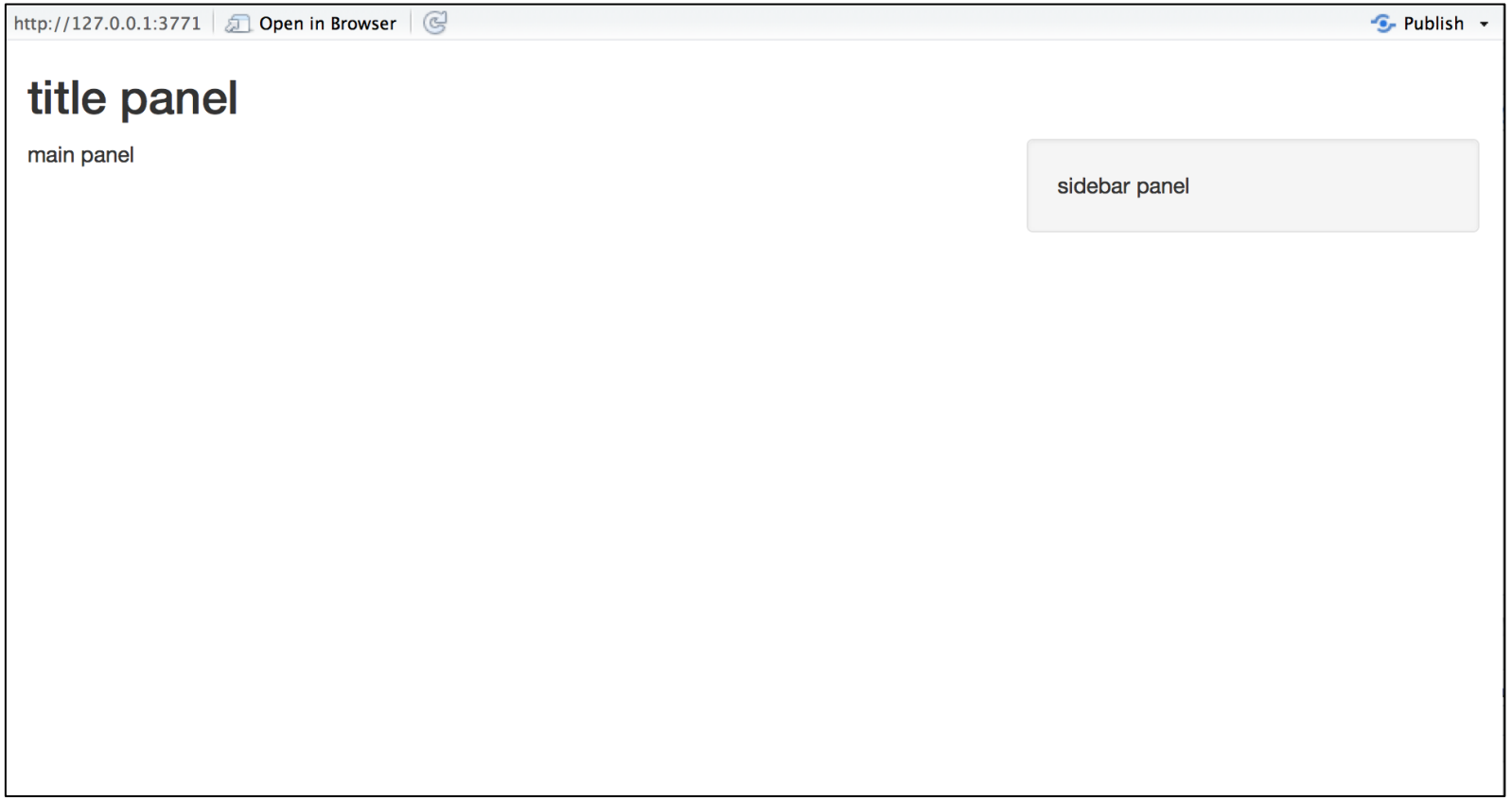
Fancier layouts are also possible.

You can also add customized text with HTML formatting.

ui - basics

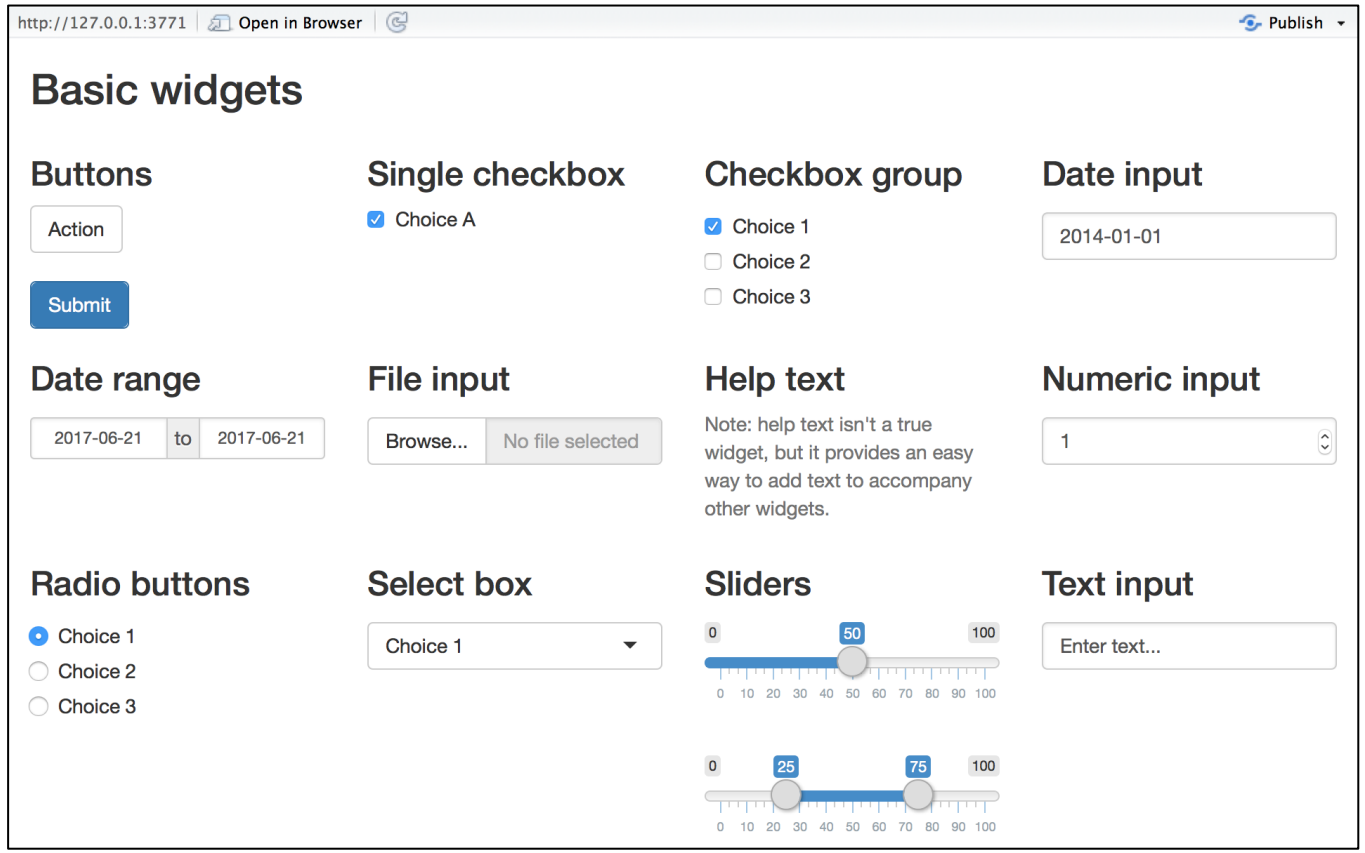
```
ui <- fluidPage(
  titlePanel("title panel"),

  sidebarLayout(position = "right",
                sidebarPanel("sidebar panel"),
                mainPanel("main panel")
  )
)
```

Combined with a server function, this is what the above ui gives you:



ui – basic widgets

Each widget is implemented with an R function. It will pass its input to the server function.



R function	widget
`actionButton`	Action Button
`checkboxGroupInput`	A group of check boxes
`checkboxInput`	A single check box
`dateInput`	A calendar to aid date selection
`dateRangeInput`	A pair of calendars for selecting a date range
`fileInput`	A file upload control wizard
`helpText`	Help text that can be added to an input form
`numericInput`	A field to enter numbers
`radioButtons`	A set of radio buttons
`selectInput`	A box with choices to select from
`sliderInput`	A slider bar
`submitButton`	A submit button
`textInput`	A field to enter text

For examples: https://shiny.rstudio.com/gallery/widget-gallery.html

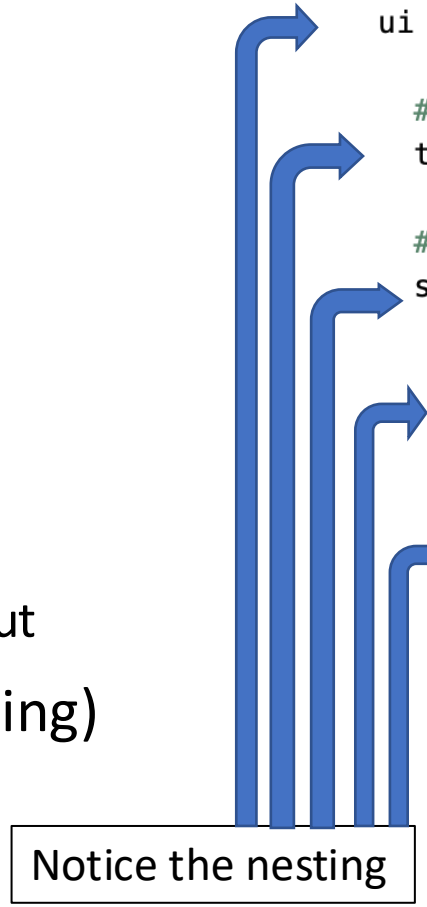
ui – adding widgets

Add a widget by calling its function within `sidebarPanel`.

The first two arguments are always

- The widget's name (a character string)
  - Will get used to access the stored user input
- A label that the user sees (character string)

Other arguments vary.




```
library(shiny)

# Define UI for app that draws a histogram ----
ui <- fluidPage(

  # App title ----
  titlePanel("Hello Shiny!"),

  # Sidebar layout with input and output definitions ----
  sidebarLayout(

    # Sidebar panel for inputs ----
    sidebarPanel(

      # Input: Slider for the number of bins ----
      sliderInput(inputId = "bins",
                  label = "Number of bins:",
                  min = 1,
                  max = 50,
                  value = 30)

    ),

    # Main panel for displaying outputs ----
    mainPanel(

      # Output: Histogram ----
      plotOutput(outputId = "distPlot")

    )
  )
)
```

Reactive output

Automatic response when the user adjusts a widget.

Two steps

1. Add an R object to the ui (let's use `mainPanel`).
2. Tell R how to build it (do this in the server function).
   It will be reactive if the code uses a user-entered widget value.

Reactive output – 1. add R object to ui

Choose the function that returns the type of object you want to show.

Name the object in the ui; build it below in the server.

The name is the only argument for the function.

Output function	Creates
`dataTableOutput`	DataTable
`htmlOutput`	raw HTML
`imageOutput`	image
`plotOutput`	plot
`tableOutput`	table
`textOutput`	text
`uiOutput`	raw HTML
`verbatimTextOutput`	text

```
library(shiny)

# Define UI for app that draws a histogram ----
ui <- fluidPage(

  # App title ----
  titlePanel("Hello Shiny!"),

  # Sidebar layout with input and output definitions ----
  sidebarLayout(

    # Sidebar panel for inputs ----
    sidebarPanel(

      # Input: Slider for the number of bins ----
      sliderInput(inputId = "bins",
                  label = "Number of bins:",
                  min = 1,
                  max = 50,
                  value = 30)

    ),

    # Main panel for displaying outputs ----
    mainPanel(

      # Output: Histogram ----
      plotOutput(outputId = "distPlot")

    )
  )
)
```

Reactive output – 2. build object within `server`

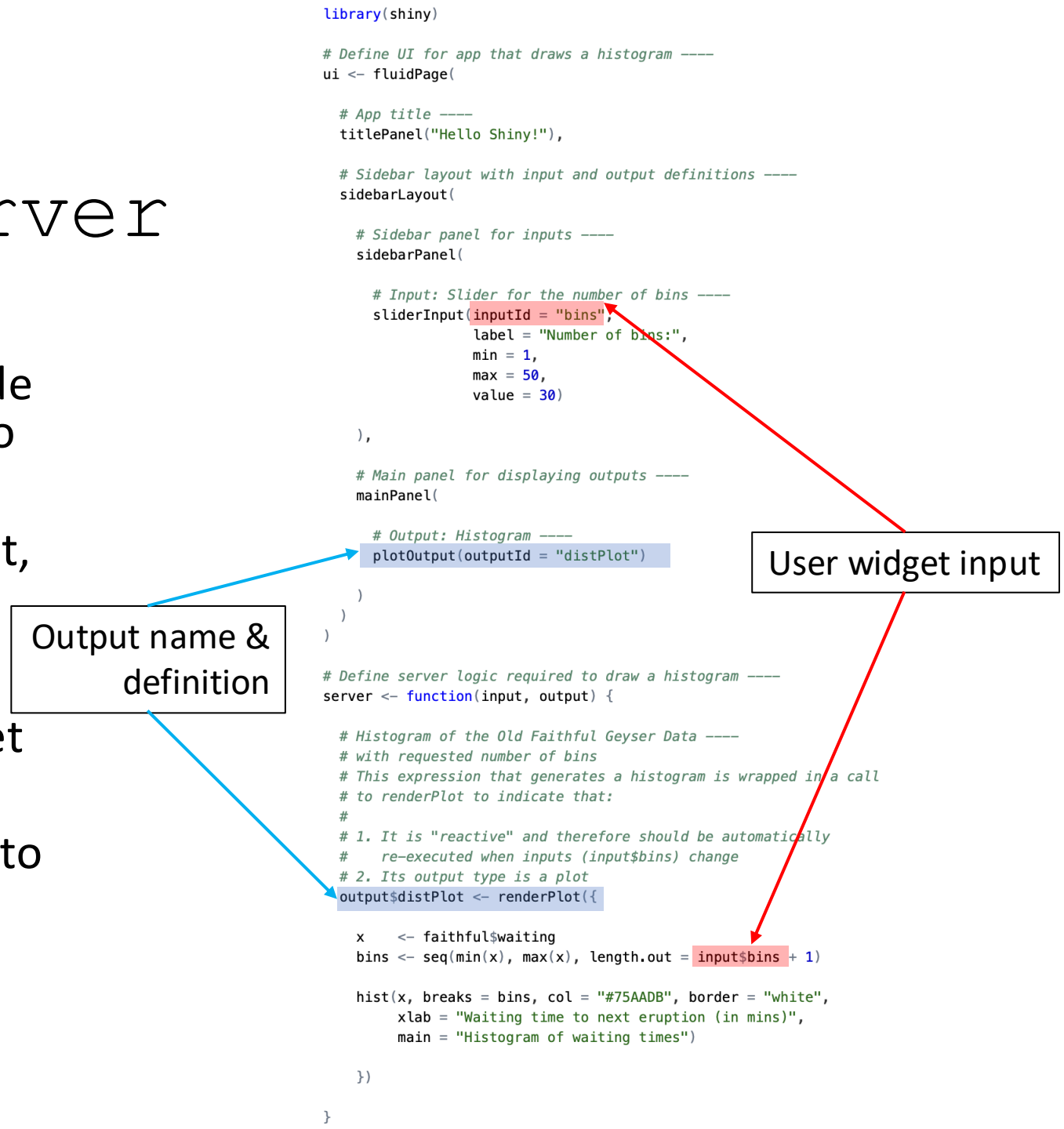
Above, in `sidebarLayout` > `sidebarPanel` > `mainPanel`, we told Shiny where to put the output.

Now we give code to build it inside the `server` function, which:

- Builds a list object named `output` containing all the code to update the object when the user enters new widget values
- Gives each object its own entry in the `output` list
- Do this by matching the entry in `output` with an object name given above as the argument to an output function

Reactive output – 2. build object within `server`

- `server` automatically returns `output`
- Each output component gets wrapped inside of a `render*` function that corresponds to the type of reactive object
- The `render*` functions take one argument, an R expression in curly braces `{ }`
- The expression can be one line or many
- Gets re-evaluated immediately when widget values change
- Need to `render*` the right type of object to match the `*Output` function above




```r
library(shiny)

# Define UI for app that draws a histogram ----
ui <- fluidPage(

  # App title ----
  titlePanel("Hello Shiny!"),

  # Sidebar layout with input and output definitions ----
  sidebarLayout(

    # Sidebar panel for inputs ----
    sidebarPanel(

      # Input: Slider for the number of bins ----
      sliderInput(inputId = "bins",
                  label = "Number of bins:",
                  min = 1,
                  max = 50,
                  value = 30)

    ),

    # Main panel for displaying outputs ----
    mainPanel(

      # Output: Histogram ----
      plotOutput(outputId = "distPlot")

    )
  )
)

# Define server logic required to draw a histogram ----
server <- function(input, output) {

  # Histogram of the Old Faithful Geyser Data ----
  # with requested number of bins
  # This expression that generates a histogram is wrapped in a call
  # to renderPlot to indicate that:
  #
  # 1. It is "reactive" and therefore should be automatically
  #    re-executed when inputs (input$bins) change
  # 2. Its output type is a plot
  output$distPlot <- renderPlot({

    x    <- faithful$waiting
    bins <- seq(min(x), max(x), length.out = input$bins + 1)

    hist(x, breaks = bins, col = "#75AADB", border = "white",
         xlab = "Waiting time to next eruption (in mins)",
         main = "Histogram of waiting times")

    })

}
```

Reactive output – 2. (continued) `render*` functions

render function	**creates**
`renderDataTable`	DataTable
`renderImage`	images (saved as a link to a source file)
`renderPlot`	plots
`renderPrint`	any printed output
`renderTable`	data frame, matrix, other table like structures
`renderText`	character strings
`renderUI`	a Shiny tag object or HTML

Reactive output – 2. (continued)
Using the widget values

All user-supplied widget values are stored in the `input` list object.

Access them with the `$` operator (see previous slide example).

`server` automatically makes any output component reactive if that component uses an `input` value, so objects stay "up-to-date."

You just need to connect `input` to `output` objects, and Shiny does the rest.

Let’s walk through a couple of examples

In RStudio

Try an example I’ve provided, `app.R`, as well as two built-in examples:

- `runApp(“path/to/data/text_entry_app/app.R”, display.mode = "showcase")`
- `runExample("01_hello")`
- `runExample("02_text")`

More built-in examples

```
runExample("01_hello")          # a histogram
runExample("02_text")           # tables and data frames
runExample("03_reactivity")     # a reactive expression
runExample("04_mpg")            # global variables
runExample("05_sliders")        # slider bars
runExample("06_tabsets")        # tabbed panels
runExample("07_widgets")        # help text and submit buttons
runExample("08_html")           # Shiny app built from HTML
runExample("09_upload")         # file upload wizard
runExample("10_download")       # file download wizard
runExample("11_timer")          # an automated timer
```

Try a simple one on your own

Try making a new copy of my provided example `text_entry_app/app.R` and modify it to

- Keep the original text entry and output
- Now have a slider input that prompts the user to enter how hungry they are, on a scale of 1-10, with a default of 4
- Output a message (text) re-stating how hungry the user said they were
- A solution is provided in `hungry/app.R`

Making a Manhattan plot

`chip_2_genesis_gwas_results.txt.gz` contains a table of GWAS results.

```
> head(d)
    variant.id chr    pos n.obs       freq MAC      Score Score.SE Score.Stat Score.pval        Est    Est.SE          PVE effect_allele other_allele strand
1:    rs806731   1  30923  1226 0.13132137 322  1.8716091 2.798572  0.6687730 0.50364029  0.2389694 0.3573251 0.0003662529             A            G      +
2: rs183605470   1  84139  1226 0.01508972  37  2.6859570 1.140658  2.3547444 0.01853545  2.0643745 0.8766873 0.0045405783             A            G      +
3: rs148663102   1 254186  1226 0.01672104  41 -0.6500653 1.284419 -0.5061161 0.61277517 -0.3940427 0.7785619 0.0002097606             A            G      +
4: rs144425991   1 529782  1226 0.08809135 216  0.8124409 2.653831  0.3061390 0.75949884  0.1153574 0.3768138 0.0000767469             A            G      +
5: rs529192661   1 564158  1226 0.02039152  50 -2.2628646 1.409511 -1.6054250 0.10840030 -1.1389941 0.7094658 0.0021105890             A            G      +
6:   rs6594031   1 565541  1226 0.10032626 246 -4.6564634 2.961961 -1.5720879 0.11593017 -0.5307591 0.3376141 0.0020238451             A            G      +
```

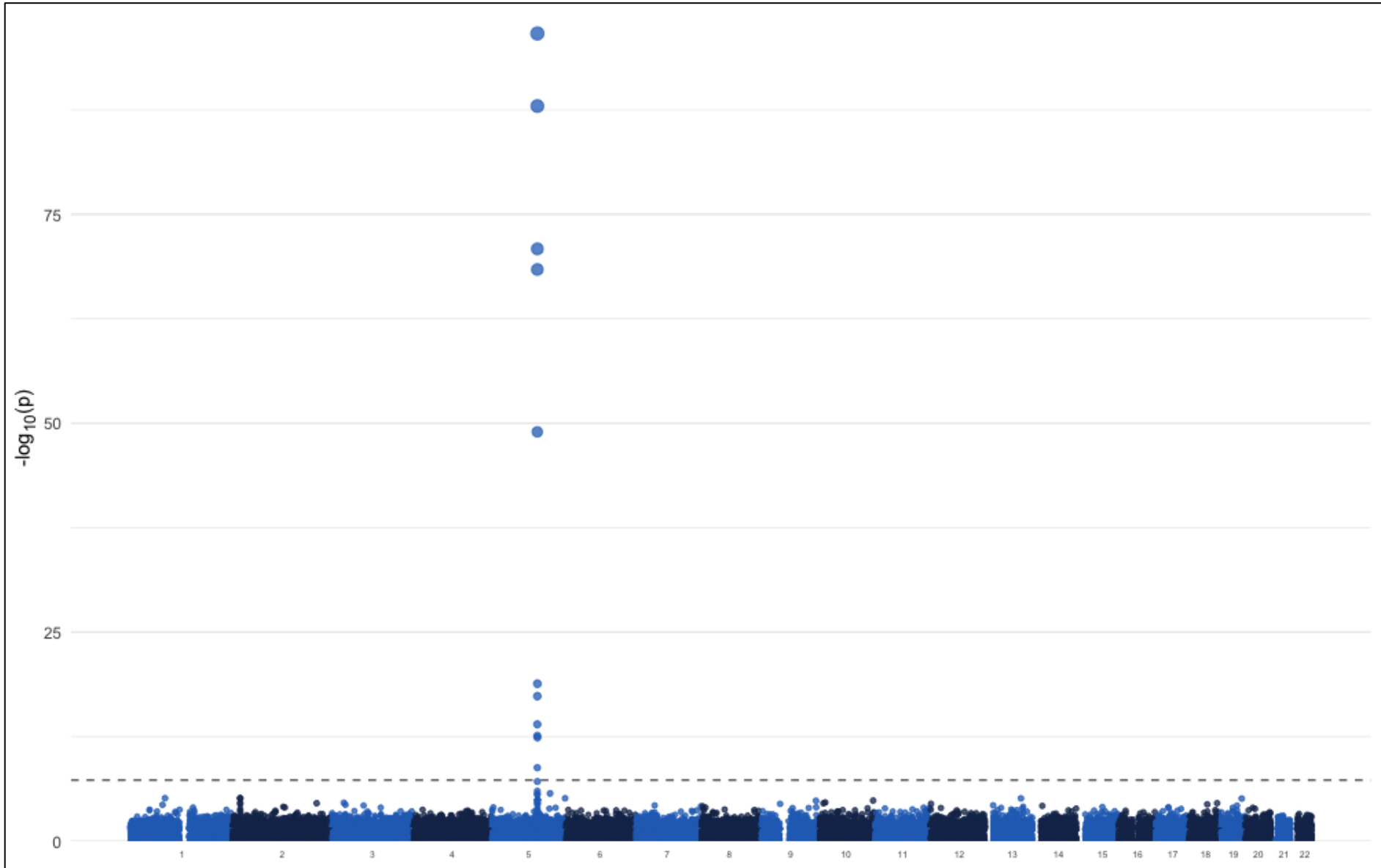
`static_manhattan_plot.R` reads in the data (you need to adjust the path and possibly install some packages) and makes a Manhattan plot.

Let's take a moment to see how the code works and see the plot.

Code adapted from https://danielroelfs.com/blog/how-i-create-manhattan-plots-using-ggplot/

Our default Manhattan plot

Not ideal!



Interactive Manhattan plot with Shiny

`interactive_manhattan_plot/app.R` is a modified version with some widgets to help us improve the plot.

- Adjust the y-axis maximum so that the plot isn't all whitespace
- Filtering non-significant p-values to speed up plotting
  - P-value threshold above which SNPs should be thinned
  - What percent of those non-significant SNPs to plot
- Adjustable genome-wide significance line

Interactive Manhattan plot with Shiny

More ideas

- Filter SNPs
  - Allele frequency
  - Allele count
  - Sample size
  - Effect size/direction
- Filter regions
  - Chromosomes
  - By annotations (if we had them in this dataset)
- Other aesthetics
  - Type in the name of a SNP to highlight it
- Output summary tables, with messages about numbers of variants filter out
- What else? See if you/we can implement one of your ideas.